

L'application est une **cartographie interactive pour la Terre du Milieu** avec une architecture modulaire basée sur des **managers spécialisés**. Elle utilise **Flask** (Python) pour le backend et **JavaScript vanilla** avec des modules ES6 pour le frontend.

Structure des Managers

1. DataManager (`js/managers/data-manager.js`)

- **Rôle** : Gestion des données de lieux et régions
- **Responsabilités** :
 - Chargement initial depuis JSON ou localStorage
 - Sauvegarde locale des lieux et régions
 - Exposition des données via `window.locationsData` et `window.regionsData`

2. FilterManager (`js/managers/filter-manager.js`)

- **Rôle** : Système de filtrage unifié pour lieux et régions
- **Responsabilités** :
 - Filtres par couleur, statut (visit /connu), type
 - Gestion du panneau de filtres
 - Mise   jour de l'affichage selon les filtres actifs

3. VoyageManager (`js/managers/voyage-manager.js`)

- **R le** : Gestion des voyages jour par jour
- **Responsabilit s** :
 - Calcul des segments de voyage
 - G n ration des donn es journali res
 - Gestion des dates de calendrier pour les voyages
 - Int gration avec GeminiManager pour g n ration de descriptions

4. PathManager (`js/managers/path-manager.js`)

- **R le** : Gestion du trac  de voyage sur la carte
- **Responsabilit s** :
 - Mode dessin activable/d sactivable
 - D tection des lieux et r gions travers s
 - Calcul des distances
 - Gestion du canvas de dessin
 - Exposition via `window.journeyPath`, `window.journeyDiscoveries`, `window.regionSegments`

5. CalendarManager (`js/managers/calendar-manager.js`)

- **R le** : Gestion du calendrier et des saisons
- **Responsabilit s** :
 - Import/export de calendriers CSV
 - Gestion des saisons et m t o
 - Synchronisation avec le syst me de voyage
 - Exposition via `window.calendarData`, `window.currentCalendarDate`, `window.isCalendarMode`, `window.currentSeason`

6. **SettingsManager** (`js/managers/settings-manager.js`)

- **Rôle** : Gestion des paramètres de l'application
- **Responsabilités** :
 - Gestion des cartes (joueur/gardien)
 - Descriptions des aventuriers et quêtes
 - Styles de narration
 - Interface de configuration

7. **AuthManager** (`js/managers/auth-manager.js`)

- **Rôle** : Authentification Google OAuth et gestion des contextes cloud
- **Responsabilités** :
 - Connexion/déconnexion Google
 - Sauvegarde/chargement de contextes utilisateur
 - Synchronisation automatique
 - Gestion de l'interface utilisateur d'authentification

8. **InfoBoxManager** (`js/managers/infobox-manager.js`)

- **Rôle** : Gestion des info-boxes pour lieux et régions
- **Responsabilités** :
 - Affichage des détails (image, texte, rumeurs, traditions)
 - Mode édition avec onglets
 - Intégration avec UploadManager et GeminiManager
 - Système multi-images

9. **ImportExportManager** (`js/managers/import-export-manager.js`)

- **Rôle** : Import/export unifié de données
- **Responsabilités** :
 - Export JSON unifié (lieux + régions)
 - Import avec validation de format
 - Modes fusion/remplacement
 - Gestion de la modal de confirmation

10. **ZoomManager** (`js/managers/zoom-manager.js`)

- **Rôle** : Contrôle du zoom avec interface visuelle
- **Responsabilités** :
 - Barre de zoom interactive
 - Boutons +/- et reset
 - Synchronisation avec le zoom molette
 - Affichage du pourcentage

11. **GeminiManager** (`js/managers/gemini-manager.js`)

- **Rôle** : Intégration avec l'API Google Gemini
- **Responsabilités** :
 - Génération de descriptions de lieux
 - Génération de récits de voyage

- Gestion des animations de chargement

12. UploadManager (`js/managers/upload-manager.js`)

- **Rôle** : Upload d'images
- **Responsabilités** :
 - Validation de fichiers
 - Upload avec progression
 - Composants drag & drop
 - Gestion des catégories (locations/regions)

Principes d'Architecture

1. Organisation Modulaire

- Chaque manager est **autonome** et gère un domaine fonctionnel spécifique
- Imports ES6 : `import ManagerName from './managers/manager-name.js'`
- **main.js** reste minimal : initialisation et coordination

2. Gestion des Variables Globales

Variables exposées globalement pour compatibilité :

```
// Données

window.locationsData // DataManager

window.regionsData // DataManager

// Calendrier

window.calendarData // CalendarManager

window.currentCalendarDate

window.isCalendarMode

window.currentSeason

// Voyage/Tracé

window.journeyPath // PathManager

window.journeyDiscoveries

window.regionSegments

window.totalPathPixels
```

```

window.lastPoint

window.isDrawingMode

// Zoom

window.scale // Variables de navigation

// Managers exposés

window.zoomManager

window.infoBoxManager

window.importExportManager

window.authManager

window.settingsManager

window.calendarManager

```

Principe de synchronisation :

- Les managers **modifient leurs données internes** en priorité
- Ils **exposent ensuite via `window`** pour la compatibilité avec le code existant
- Les fonctions globales comme `renderLocations()` et `renderRegions()` sont exposées pour permettre les mises à jour

3. Constantes Centralisées

- Fichier `js/utils/constants.js` contient :
 - Couleurs (`colorMap`, `regionColorMap`)
 - URLs de cartes et données
 - Symboles de saisons
 - Configurations par défaut

4. Communication entre Managers

- **Injection de dépendances** : passage de managers ou callbacks au constructeur
- **Variables globales** : pour partage de données d'état
- **Événements DOM personnalisés** : `document.dispatchEvent(new CustomEvent(...))`

5. Cycle de vie typique

```
// 1. Initialisation dans main.js
```

```
const manager = new ManagerName(domUtils, dependencies);

// 2. Configuration

manager.init();

// 3. Exposition globale si nécessaire

window.managerName = manager;

// 4. Le manager maintient son état interne et expose des données via
window
```

Backend (Flask)

- **app.py** : Routes API, authentification OAuth, upload d'images
- **database.py** : Gestion SQLite pour contextes utilisateur
- Endpoints principaux :
 - `/api/auth/*` : Authentification Google
 - `/api/contextes/*` : CRUD des contextes de voyage
 - `/api/gemini/*` : Génération de contenu IA
 - `/api/upload/*` : Upload d'images

Principe de Développement

"Un manager = une responsabilité" :

- Limite la taille de `main.js`
- Facilite la maintenance
- Permet la réutilisation
- Respecte le principe de séparation des préoccupations